Embedded Data Structure
(Lecture Note 23)

Searching Algorithms (6)

2025. 6. 5

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class

- **Binary tree search**
  - **How can we sort data in the binary tree searching scheme ?**
    - Think about the inorder traverse (why inorder traverse ?)
  - **Make a tree balanced**
    - Inorder traverse and recursive operation

    ➔ **The searching performance is efficiently improved**

Introduction (1/3)

KU KONKUK UNIVERSITY

▪ Counting words in a given text file

- **Counts a key word using the binary tree search**

  - How can we count a specific word by searching algorithm ?

  (※ Think about the return value of the bti_search function)

  1) If the return value is NULL, then ?
  2) If the return value is not NULL, then ?

- **Try it once again with a balanced tree**

c has only one-dimensional arrays, and the size of an array must be fixed as a constant at compilation time. However, an element of an array may be an object of any type, including another array; this makes it possible to simulate multi-dimensional array fairly easily.

[Example of test.txt file]

KU KONKUK UNIVERSITY

▪ Counting words in a given text file – cont'd

- **First of all, make a new structure for words**

```
typedef struct _wcount{
    int count;          ← To store "counts" for a key word
    char word[20];      ← To store a key word
}wcount;

int wcount_cmp(const void *a, const void *b)
{
    return strcmp(((wcount *)a)->word, ((wcount *)b)->word);
}   ← If two strings are same, then return is ...

void print_wcount(void *a)   ← Print a key word and its counts
{
    printf("%s : %d₩n", ((wcount *)a)->word, ((wcount *)a)->count);
}
```

Introduction (3/3)

KU KONKUK UNIVERSITY

▪ Counting words in a given text file – cont'd

- **To divide a string into words, we employ "strtok()" function**
  - To use strtok() function, we need to include "string.h" header
  - We use the "delim" to store our separators as follows:

    (Example) char delim[] = " ₩t₩r₩n.,-";
  - "strtok()" function **adds NULL character** for the next division !

```
void main(){
    char s[] = "All the things you are";
    char delim[] = " ,.:";
    char *token;

    token = strtok(s, delim);
    while(token != NULL){
        token = stkrok(NULL, delim);
    }
}
```

For next division !

Technical Details (1/8)

▪ Implementation (baseline)

- **Before starting to code the main function,**

→ Implement generalized versions of binary tree searching functions !

```
#include <stdio.h>
#include <stdlib.h>
#include <Windows.h>

typedef int(*FCMP)(const void *, const void*);

typedef struct _node {
   struct _node *left;
   struct _node *right;
}node;

void init_tree(node **p);
void *btv_search(void *key, node *base, int *num, int width, FCMP fcmp);
void *btv_insert(void *key, node *base, int *num, int width, FCMP fcmp);
void btv_sort_list(node *p, void(*fptr)(void *));
```

BTS_library.h

You need to make

BTS_library.cpp

KU KONKUK UNIVERSITY

Implementation (baseline)

Main function

```
#include <string.h>
#include "BTS_library.h"

typedef struct _wcount{
   int count;
   char word[20];
}wcount;

node *head;
const char delim[] = " ₩t₩r₩n,.-";

int wcount_cmp(const void *a, const void *b)
{
   return strcmp( ((wcount *)a)->word, ((wcount *)b)->word );
}

void print_wcount(void *a)
{
   printf("%s : %d₩n", ((wcount *)a)->word, ((wcount *)a)->count);
}
```

```
void main()
{
   int i, num = 0, cnt = 0;;

   char *tok;
   char str[256];
   wcount wc, *wcp;

   init_tree(      );

   FILE *fp;
   fp = fopen("test.txt", "r");

   fgets(str, 256, fp); // read one line
   while (!feof(fp)){
      Part 1
   }
   Part 2
}
```

Technical Details (3/8)

- **Implementation (baseline) – cont'd**
  - **Main function (continue)**

```
tok = strtok(str, delim);
while (tok){
   strcpy(        , tok);
   wcp = (wcount *)btv_search(    , head, &num, sizeof(wcount), wcount_cmp);

   if (wcp == NULL) {
      wc.count = 1; // count frequency
      btv_insert(    , head, &num, sizeof(wcount), wcount_cmp);
   }
   else
      ;
   tok = strtok(NULL, delim);
}
fgets(str, 256, fp);
```

Part 1

```
printf("₩n");
btv_sort_list(        , print_wcount);
```

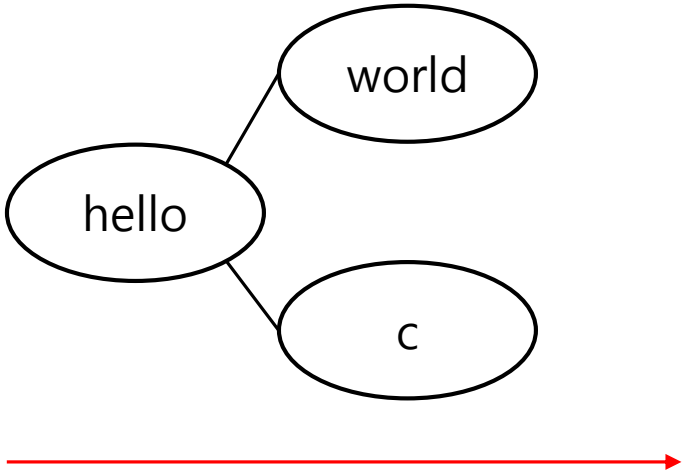
You can check results !

Part 2

Implementation (Advanced)

- **Print the tree as the text in the command window**

```
void btv_list(node *p, void(*fptr)(void *))
{
    int i;
    static int x = 0;

    if (p != NULL)
    {
        x += 2; // position for displaying a node

        // a kind of inorder traverse
        btv_list(p->right, fptr);
        for (i = 2; i < x; i++) printf("  ");
            fptr(p + 1); // print it in the monitor
        btv_list(p->left, fptr);
        x -= 2;
    }
}
```



Display text like this !

KU KONKUK UNIVERSITY

▪ Implementation (Advanced)

- **Print the tree as the text in the command window**

c has only one-dimensional arrays, and the size of an array must be fixed as a constant at compilation time. However, an element of an array may be an object of any type, including another array; this makes it possible to simulate multi-dimensional array fairly easily.

```
                    must : 1
                            multi : 1
                        may : 1
                                makes : 1
                                    it : 1
                            including : 1
    has : 1
            fixed : 1
                    fairly : 1
                element : 1
                    easily : 1
        dimensional : 2
            constant : 1
                compilation : 1
c : 1
        be : 2
                at : 1
            as : 1
    arrays : 1
                array; : 1
            array : 3
                any : 1
                    another : 1
        and : 1
            an : 4
                a : 1
                    However : 1
```

Technical Details (6/8)

▪ Implementation (Advanced) – cont'd

- **Draw the node graphically, download "BTS.zip" from class web**

→ Check three functions (draw_node, draw_link, btv_draw)

→ Also, you need to download the **"LectureNote_23_supplements.zip"**)

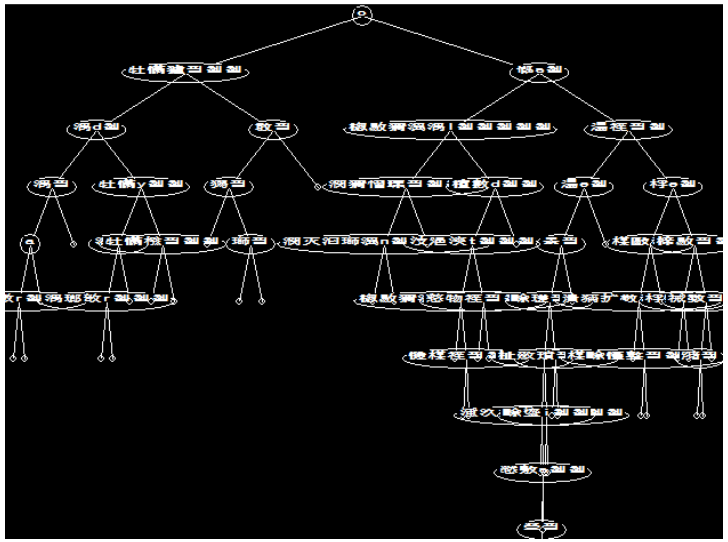
(Add "winbgi.cpp" and "graphics.h")

Usage :

```
btv_draw(head->left, make_str, LINK | NODE | TAIL);
```

where

```
void make_str(void *a, char *s)
{
   strcpy(s, ((wcount *)a)->word);
}
```



Result of binary tree

KU KONKUK UNIVERSITY

Implementation (Advanced) – cont'd

- **Make a tree balanced**

  - You need to make generalized versions of two functions as follows:

```
void _sort(node *p, void *a, int width);
node *_balance(int n, void *a, int width);
```

⇨

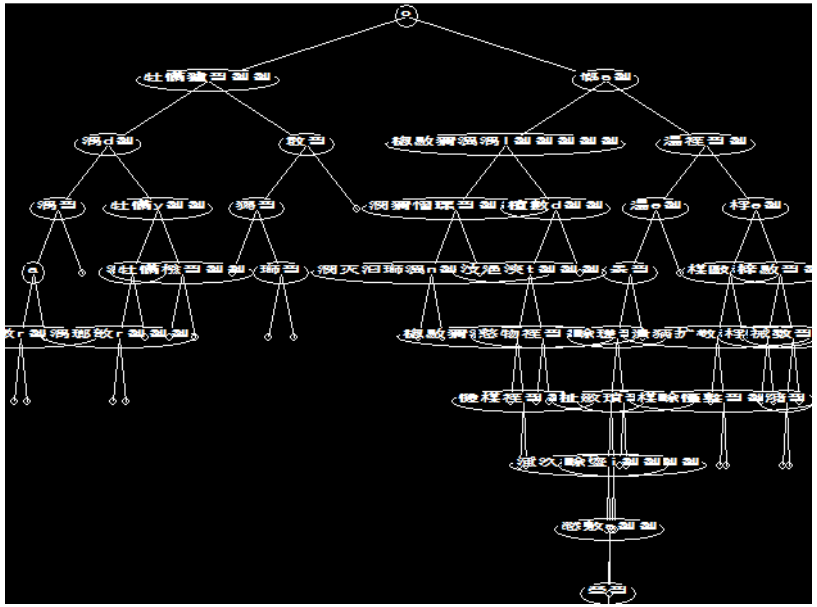
```
void btv_balance(node *base, int *num, int width);
```

```
void btv_balance(node *base, int *num, int width)
{
    void *tmp;
    int ntmp = *num;
    tmp = malloc(width*ntmp);
    _index = 0;
    _sort(base->left, tmp, width);
    ntmp = *num;
    btv_deleteall(base, num);
    _index = 0;
    base->left = _balance(ntmp, tmp, width);
    *num = ntmp;
    free(tmp);
}
```

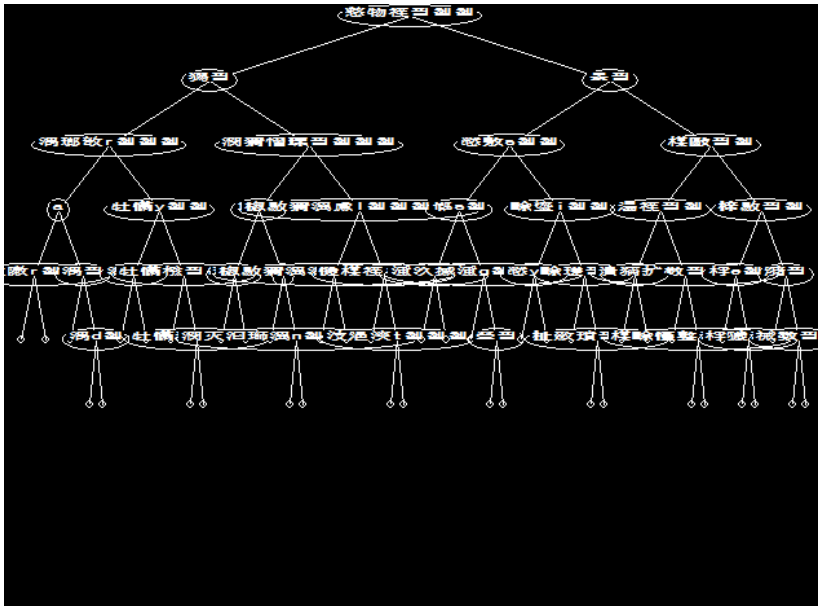
```
void _deleteall(node *t)
{
    if (t != NULL){
        _deleteall(t->left); _deleteall(t->right); free(t);
    }
}

void *btv_deleteall(node *base, int *num)
{
    node *t;
    t = base->left;
    _deleteall(t);
    base->left = NULL;
    base->right = NULL;
    *num = 0;
    return base;
}
```

Technical Details (8/8)

- **Implementation (Advanced) – cont'd**
  - **Check the result of the balanced tree**



Result of binary tree



Result of balanced binary tree

Summary

- **Make a tree balanced !**
  - **You can count a key word in your text file based on BTS**
  - **Try to COMPLETELY implement today's topic**